Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información
Centro Regional de Mazatenango
Curso: Análisis y Diseño de Sistemas II
Catedrático: Ing. Angel Atilio Maltez C.

Manual Técnico

Sistema de Gestión del Asilo de Ancianos «Cabeza de Algodón»

Instalación, arquitectura, base de datos, API y código · versión 3.0

Gilmer Iván Ortiz Samayoa
Carné: 3090-23-12685

Mazatenango, Suchitepéquez, 30 de octubre de 2026

Contenido

1. Introducción 3
2. Arquitectura 4
    2.1 Tecnologías y versiones 4
3. Estructura del proyecto 5
4. Instalación 6
    4.1 Requisitos 6
    4.2 Instalación local con XAMPP (instancia de la base de datos) 6
    4.3 Instalación en hosting 6
    4.4 Modo demostración 7
5. Base de datos 8
    5.1 Diccionario de datos 9
    5.2 Vistas y procedimientos 15
6. API del servidor (api/api.php) 17
    6.1 Permisos por tabla en el servidor 17
7. Documentación del código 18
    7.1 js/servicios.js · lógica de negocio 18
    7.2 js/auth.js · seguridad y sesión 19
    7.3 js/datos.js · acceso a datos 20
    7.4 api/api.php · servidor 21
    7.5 js/app.js · presentación 22
8. Seguridad 26
9. Pruebas automatizadas 27
10. Mantenimiento 28
    10.1 Solución de problemas técnicos 28
11. Control de versiones 29

1. Introducción

Este manual técnico describe cómo está construido el Sistema de Gestión del Asilo de Ancianos «Cabeza de Algodón», cómo se instala, cómo funciona cada componente del código y cómo se mantiene. Está dirigido al personal técnico que instalará, revisará o dará mantenimiento al sistema, y al analista SQA.

Dato	Valor
Versión del sistema	3.0 (línea base ERS v3.0, rev. 1 del 30/10/2026)
Repositorio	https://github.com/I-labGT/sistema-asilo-cabeza-de-algodon
Sistema en línea	https://asilo-cabeza-algodon.site.je (InfinityFree, PHP 8 + MySQL, HTTPS)
Responsable	Gilmer Iván Ortiz Samayoa · carné 3090-23-12685
Licencias	Todo el software usado es gratuito (PHP, MySQL/MariaDB, Apache, XAMPP)

2. Arquitectura

Cliente-servidor en tres capas: **presentación** (HTML, CSS y JavaScript en el navegador), **lógica de negocio** (js/servicios.js en el cliente y api/api.php en el servidor) y **datos** (MySQL o MariaDB). La página se comunica con el servidor mediante peticiones HTTP con cuerpo JSON a un único punto de entrada.

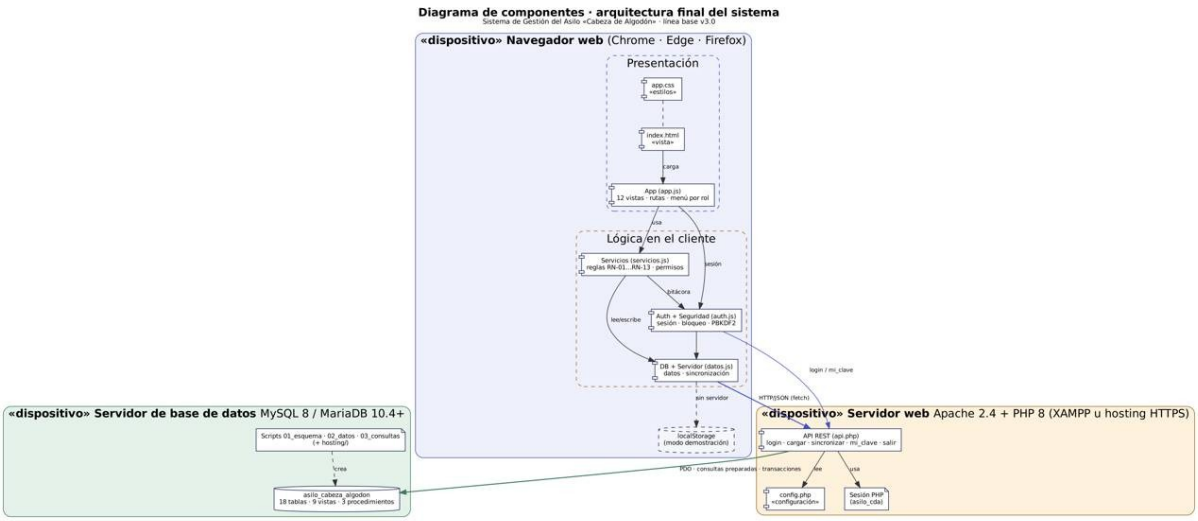


Figura 1. Diagrama de componentes de la arquitectura final

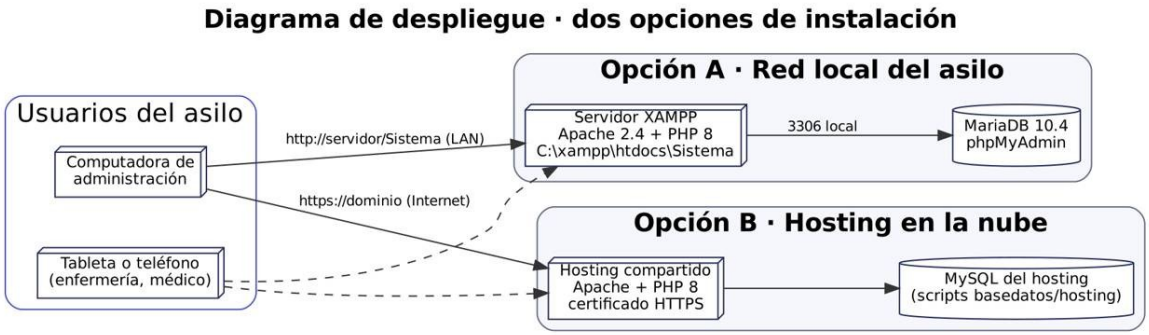


Figura 2. Diagrama de despliegue

2.1 Tecnologías y versiones

Componente	Tecnología	Versión mínima
Navegador	Chrome, Edge o Firefox (con WebCrypto)	Dos últimas versiones
Servidor web	Apache	2.4
Lenguaje del servidor	PHP con PDO MySQL	8.0
Base de datos	MySQL o MariaDB (InnoDB, utf8mb4)	MySQL 8 / MariaDB 10.4
Paquete local	XAMPP (Apache + PHP + MariaDB + phpMyAdmin)	8.x
Pruebas	Python + Playwright	Python 3.10

3. Estructura del proyecto

Ruta	Contenido
Sistema/index.html	Estructura: pantalla de inicio de sesión, cambio de contraseña y marco de la aplicación
Sistema/css/app.css	Estilos, variables de color, modo oscuro, impresión y responsividad
Sistema/js/app.js	Capa de presentación: rutas, menú por rol, 12 vistas, formularios, reportes
Sistema/js/servicios.js	Reglas de negocio y permisos por rol
Sistema/js/auth.js	Seguridad: sesión, bloqueo e hash PBKDF2
Sistema/js/datos.js	Acceso a datos: modo servidor o demostración, sincronización
Sistema/api/api.php	API del servidor
Sistema/api/config.php	Datos de conexión a MySQL
Sistema/basedatos/01_esquema.sql	Crea la base y las 18 tablas
Sistema/basedatos/02_datos.sql	Datos ficticios iniciales
Sistema/basedatos/03_consultas.sql	9 vistas y 3 procedimientos
Sistema/basedatos/hosting/	Los mismos scripts sin CREATE DATABASE, para hosting
Evidencias/pruebas/	Pruebas automatizadas prueba_sqa.py y prueba_cp.py
Documentacion/	ERS, manuales, planes, diagramas y capturas

4. Instalación

4.1 Requisitos

- Equipo servidor: Windows 10/11 o Linux, 4 GB de RAM, 1 GB libre en disco.
- Equipos cliente: cualquier computadora, tableta o teléfono con navegador actualizado.
- Red local o Internet (para hosting, certificado HTTPS).

4.2 Instalación local con XAMPP (instancia de la base de datos)

1. Descargar XAMPP con PHP 8 desde https://www.apachefriends.org e instalarlo con Apache, MySQL y phpMyAdmin.
2. Abrir el XAMPP Control Panel y pulsar **Start** en Apache y en MySQL.
3. Copiar la carpeta **Sistema** a **C:\xampp\htdocs\Sistema**.
4. Abrir **http://localhost/phpmyadmin** › pestaña **Importar** y cargar, en este orden: **01_esquema.sql** (crea la instancia asilo_cabeza_algodon y sus 18 tablas), **02_datos.sql** (datos iniciales) y **03_consultas.sql** (vistas y procedimientos).
5. Comprobar en phpMyAdmin que existen 18 tablas, 9 vistas y 3 procedimientos.
6. Revisar **Sistema/api/config.php** (por defecto: host 127.0.0.1, puerto 3306, usuario root sin contraseña, base asilo_cabeza_algodon).
7. Abrir **http://localhost/Sistema/**. Si en el pie de Usuarios aparece «Base de datos MySQL conectada», la instalación es correcta.
8. Entrar como administrador con la contraseña temporal entregada en el archivo confidencial; el sistema obliga a crear una contraseña personal.
9. Para que otros equipos de la red entren, usar http://<IP-del-servidor>/Sistema/ y permitir el puerto 80 en el firewall.

4.3 Instalación en hosting

1. Contratar un hosting con PHP 8, MySQL y certificado HTTPS.
2. Crear una base de datos y un usuario desde el panel del hosting.
3. En phpMyAdmin del hosting importar los archivos de **basedatos/hosting/**: 01_esquema, 02_datos, 03_vistas y, si el hosting lo permite, 04_procedimientos_opcional.
4. Subir el contenido de Sistema (sin la carpeta basedatos) a public_html o htdocs.
5. Editar api/config.php con el host, base, usuario y contraseña del hosting.
6. Forzar HTTPS con un archivo .htaccess en la raíz (redirección 301 de http a https) y agregar Options -Indexes.
7. Probar el inicio de sesión y cambiar las contraseñas temporales.

> En InfinityFree gratuito no se pueden crear vistas ni procedimientos (#1142, #1370): omita 03_vistas.sql y 04_procedimientos_opcional.sql si dan error; el sistema no los usa. No suba la carpeta basedatos al hosting. Instalación real: https://asilo-cabeza-algodon.site.je

4.4 Modo demostración

El modo demostración se usa **solo si se pide de forma explícita** (H-19): al abrir Sistema/index.html como archivo (file://) o al agregar **?demo=1** a la dirección. Trabaja con datos ficticios en el navegador (localStorage), muestra el aviso «Modo demostración» y usa cuentas propias cuyas contraseñas son distintas de las del servidor (se entregan aparte). Por http/https, si la API no responde, la página muestra «Sin conexión con el servidor» y no cambia a datos locales.

5. Base de datos

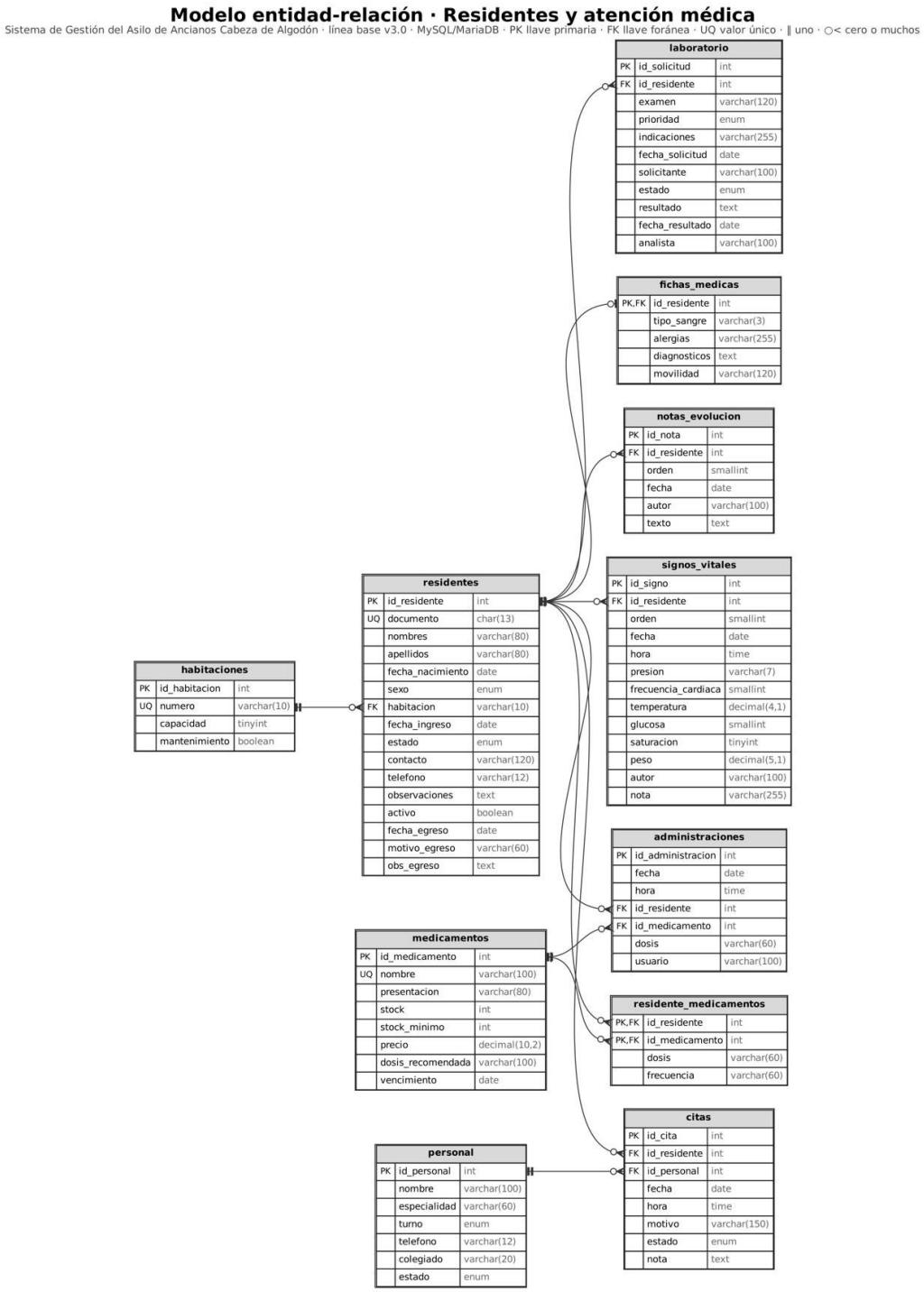


Figura 3. Modelo ER · residentes y atención médica

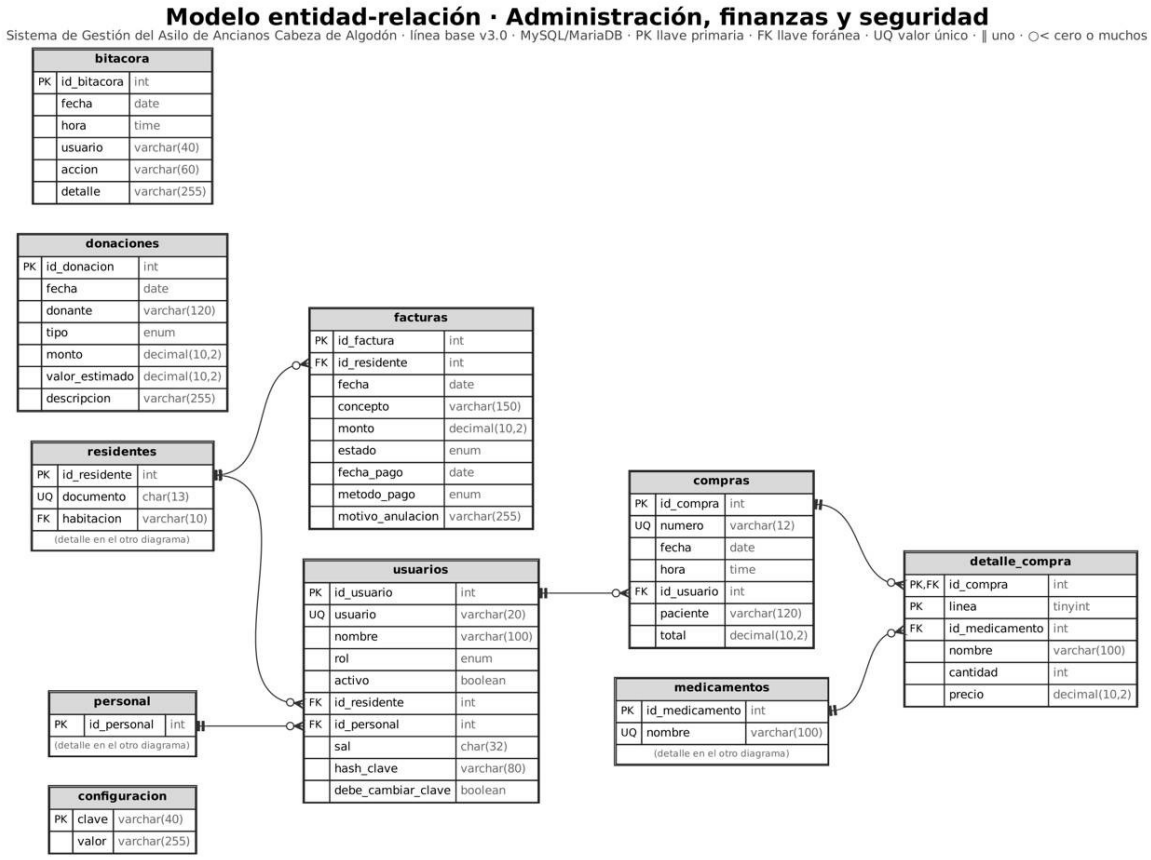


Figura 4. Modelo ER · administración, finanzas y seguridad

5.1 Diccionario de datos

configuracion

Campo	Tipo	Nulo	Llave
clave	varchar(40)	No	PK
valor	varchar(255)	No	—

habitaciones

Campo	Tipo	Nulo	Llave
id_habitacion	int	No	PK
numero	varchar(10)	No	UQ
capacidad	tinyint	No	—
mantenimiento	boolean	No	—

residentes

Campo	Tipo	Nulo	Llave
id_residente	int	No	PK
documento	char(13)	No	UQ
nombres	varchar(80)	No	—
apellidos	varchar(80)	No	—
fecha_nacimiento	date	No	—
sexo	enum	No	—
habitacion	varchar(10)	No	FK → habitaciones.numero (cascada)
fecha_ingreso	date	No	—
estado	enum	No	—
contacto	varchar(120)	Sí	—
telefono	varchar(12)	Sí	—
observaciones	text	Sí	—
activo	boolean	No	—
fecha_egreso	date	Sí	—
motivo_egreso	varchar(60)	Sí	—
obs_egreso	text	Sí	—

fichas_medicas

Campo	Tipo	Nulo	Llave
id_residente	int	No	PK · FK → residentes.id_residente (cascada)
tipo_sangre	varchar(3)	Sí	—
alergias	varchar(255)	Sí	—
diagnosticos	text	Sí	—
movilidad	varchar(120)	Sí	—

notas_evolucion

Campo	Tipo	Nulo	Llave
id_nota	int	No	PK
id_residente	int	No	FK → residentes.id_residente (cascada)
orden	smallint	No	—
fecha	date	No	—

Campo	Tipo	Nulo	Llave
autor	varchar(100)	No	—
texto	text	No	—

signos_vitales

Campo	Tipo	Nulo	Llave
id_signo	int	No	PK
id_residente	int	No	FK → residentes.id_residente (cascada)
orden	smallint	No	—
fecha	date	No	—
hora	time	No	—
presion	varchar(7)	Sí	—
frecuencia_cardiaca	smallint	Sí	—
temperatura	decimal(4,1)	Sí	—
glucosa	smallint	Sí	—
saturacion	tinyint	Sí	—
peso	decimal(5,1)	Sí	—
autor	varchar(100)	No	—
nota	varchar(255)	Sí	—

personal

Campo	Tipo	Nulo	Llave
id_personal	int	No	PK
nombre	varchar(100)	No	—
especialidad	varchar(60)	No	—
turno	enum	No	—
telefono	varchar(12)	Sí	—
colegiado	varchar(20)	Sí	—
estado	enum	No	—

medicamentos

Campo	Tipo	Nulo	Llave
id_medicamento	int	No	PK

Campo	Tipo	Nulo	Llave
nombre	varchar(100)	No	UQ
presentacion	varchar(80)	No	—
stock	int	No	—
stock_minimo	int	No	—
precio	decimal(10,2)	No	—
dosis_recomendada	varchar(100)	Sí	—
vencimiento	date	Sí	—

residente_medicamentos

Campo	Tipo	Nulo	Llave
id_residente	int	No	PK · FK → residentes.id_residente (cascada)
id_medicamento	int	No	PK · FK → medicamentos.id_medicamento (cascada)
dosis	varchar(60)	No	—
frecuencia	varchar(60)	No	—

administraciones

Campo	Tipo	Nulo	Llave
id_administracion	int	No	PK
fecha	date	No	—
hora	time	No	—
id_residente	int	No	FK → residentes.id_residente (cascada)
id_medicamento	int	No	FK → medicamentos.id_medicamento (cascada)
dosis	varchar(60)	No	—
usuario	varchar(100)	No	—

citas

Campo	Tipo	Nulo	Llave
id_cita	int	No	PK
id_residente	int	No	FK → residentes.id_residente (cascada)

Campo	Tipo	Nulo	Llave
id_personal	int	Sí	FK → personal.id_personal (nulo)
fecha	date	No	—
hora	time	No	—
motivo	varchar(150)	No	—
estado	enum	No	—
nota	text	Sí	—

laboratorio

Campo	Tipo	Nulo	Llave
id_solicitud	int	No	PK
id_residente	int	No	FK → residentes.id_residente (cascada)
examen	varchar(120)	No	—
prioridad	enum	No	—
indicaciones	varchar(255)	Sí	—
fecha_solicitud	date	No	—
solicitante	varchar(100)	No	—
estado	enum	No	—
resultado	text	Sí	—
fecha_resultado	date	Sí	—
analista	varchar(100)	Sí	—

facturas

Campo	Tipo	Nulo	Llave
id_factura	int	No	PK
id_residente	int	No	FK → residentes.id_residente (cascada)
fecha	date	No	—
concepto	varchar(150)	No	—
monto	decimal(10,2)	No	—
estado	enum	No	—
fecha_pago	date	Sí	—
metodo_pago	enum	Sí	—
motivo_anulacion	varchar(255)	Sí	—

donaciones

Campo	Tipo	Nulo	Llave
id_donacion	int	No	PK
fecha	date	No	—
donante	varchar(120)	No	—
tipo	enum	No	—
monto	decimal(10,2)	No	—
valor_estimado	decimal(10,2)	No	—
descripcion	varchar(255)	Sí	—

usuarios

Campo	Tipo	Nulo	Llave
id_usuario	int	No	PK
usuario	varchar(20)	No	UQ
nombre	varchar(100)	No	—
rol	enum	No	—
activo	boolean	No	—
id_residente	int	Sí	FK → residentes.id_residente (nulo)
id_personal	int	Sí	FK → personal.id_personal (nulo)
sal	char(32)	No	—
hash_clave	varchar(80)	No	—
debe_cambiar_clave	boolean	No	—

bitacora

Campo	Tipo	Nulo	Llave
id_bitacora	int	No	PK
fecha	date	No	—
hora	time	No	—
usuario	varchar(40)	No	—
accion	varchar(60)	No	—
detalle	varchar(255)	Sí	—

compras

Campo	Tipo	Nulo	Llave
id_compra	int	No	PK
numero	varchar(12)	No	UQ
fecha	date	No	—
hora	time	No	—
id_usuario	int	Sí	FK → usuarios.id_usuario (nulo)
paciente	varchar(120)	No	—
total	decimal(10,2)	No	—

detalle_compra

Campo	Tipo	Nulo	Llave
id_compra	int	No	PK · FK → compras.id_compra (cascada)
linea	tinyint	No	PK
id_medicamento	int	Sí	FK → medicamentos.id_medicamento (nulo)
nombre	varchar(100)	No	—
cantidad	int	No	—
precio	decimal(10,2)	No	—

5.2 Vistas y procedimientos

Objeto	Uso
v_residentes_activos	Residentes activos con edad, habitación y datos clínicos
v_egresos	Historial de egresos con días de estancia
v_ocupacion_habitaciones	Camas ocupadas y libres por habitación
v_alertas_inventario	Stock bajo, vencidos y por vencer (60 días)
v_medicacion_hoy	Dosis asignadas y registradas hoy
v_agenda_citas	Citas con residente y profesional
v_cuentas_por_cobrar	Saldo pendiente por residente
v_ingresos_mensuales	Facturas pagadas, donaciones y farmacia por mes
v_panel_resumen	Indicadores del panel (RF-13)
sp_generar_mensualidades(p_monto)	Genera la mensualidad del mes sin duplicar (RN-08)
sp_registrar_egreso(p_residente, p_fecha, p_motivo, p_obs)	Baja lógica con sus efectos (RN-03, RN-04)

Objeto	Uso
sp_expediente(p_residente)	Expediente completo de un residente

6. API del servidor (api/api.php)

Acción	Método	Entrada	Respuesta / códigos	Acceso
estado	GET	—	{ok, bd} · 200	Público
publico	GET	—	Cifras de la pantalla de inicio	Público
login	POST	{usuario, clave}	200 {usuario, cambiarClave} · 401 datos incorrectos · 429 bloqueo	Público
verificar	POST	{clave}	Confirma la contraseña del usuario en sesión	Con sesión
mi_clave	POST	{actual, nueva}	200 · 400 política no cumplida	Con sesión (también temporal)
cargar	GET	—	Datos filtrados por rol (función filtrarPorRol) + seq con el siguiente id global de cada tabla · 401 sin sesión · 403 contraseña temporal	Con sesión
sincronizar	POST	{cambios: {tabla: {upsert:[], delete:[]}}}	200 · 403 tabla no permitida, registro ajeno (validarEscrituraRol), borrar residente o bitácora · 422 regla de negocio o registro con historial	Según PERMISOS y rol
vaciar	POST	—	Restablece datos iniciales; conserva las cuentas de usuario	Administrador
salir	POST	—	Destruye la sesión	Con sesión

6.1 Permisos por tabla en el servidor

Rol	Tablas que puede modificar
Administrador	Todas
Médico	residentes (ficha, notas, signos), citas, laboratorio, administraciones, bitacora
Laboratorio	laboratorio, bitacora
Paciente	compras, medicamentos (descuento de stock), bitacora

> Los usuarios que no son administradores no pueden modificar cuentas: su contraseña solo cambia con la acción mi_clave. Ningún rol que no sea Administrador puede eliminar registros, y nadie puede eliminar residentes.

7. Documentación del código

Cada archivo tiene un encabezado que explica su responsabilidad, y cada función tiene un comentario /** … */ que describe qué hace y qué requerimiento o regla cumple. A continuación se listan todas las funciones.

7.1 js/servicios.js · lógica de negocio

Función	Qué hace
ok()	Resultado exitoso uniforme { ok: true, data }.
err()	Resultado de error uniforme { ok: false, error }.
s()	Atajo a la sesión actual.
modulosDe()	Módulos que puede ver un rol (RNF-09).
puede()	Indica si el rol de la sesión puede realizar una acción.
sinPermiso()	Devuelve error si el rol no tiene permiso para la acción.
bitacora()	Registra una acción del usuario en la bitácora.
txt()	Normaliza un valor a texto sin espacios sobrantes.
hoy()	Fecha de hoy (AAAA-MM-DD).
edad()	Calcula la edad a partir de la fecha de nacimiento.
nombreResidente()	Nombre completo de un residente por id.
nombrePersonal()	Nombre de un profesional por id.
residentesActivos()	Lista de residentes activos (sin egreso).
residenteActivo()	Devuelve el residente si está activo.
ocupacionHabitacion()	Cuenta camas ocupadas en una habitación (RN-02).
vencimiento()	Estado de vencimiento de un medicamento: Vencido, Por vencer (≤ 60 días) o vacío (RF-10).
registrarEgreso()	Registra el egreso (baja lógica) con sus efectos: libera cama, cancela citas, elimina solicitudes no iniciadas y desactiva el usuario Paciente (RF-06, RN-03, RN-04).
registrarSignos()	Registra signos vitales y detecta valores fuera de rango (RF-16).
dosisDelDia()	Dosis asignadas para hoy con su estado (RF-17).
administrarDosis()	Registra una dosis dada con fecha, hora y usuario (RF-17).
guardarResidente()	Valida y guarda un residente nuevo o editado: DPI, edad ≥ 60, fechas, teléfono y habitación con cama libre (RF-03, RF-05, RN-01, RN-02).
eliminarResidente()	Siempre rechaza: los residentes no se eliminan (RN-03, H-05).
guardarFicha()	Guarda la ficha médica (RF-15).
agregarNota()	Agrega una nota de evolución (mínimo 5 caracteres) con fecha y autor (RF-15).
asignarMedicamento()	Asigna un medicamento con dosis y frecuencia a un residente (RF-17).
quitarMedicamento()	Quita un medicamento asignado.
crearCita()	Programa una cita validando horario 07:00–18:00, fecha y choques (RF-09, RN-05).

Función	Qué hace
reprogramarCita()	Cambia fecha u hora de una cita con las mismas validaciones (RF-09).
cambiarEstadoCita()	Avanza el estado de una cita según las transiciones permitidas (RF-09).
guardarPersonal()	Valida y guarda un profesional (RF-08).
eliminarPersonal()	Elimina un profesional si no tiene citas pendientes (RF-08).
solicitarExamen()	Crea una solicitud de laboratorio (RF-18).
iniciarExamen()	Pasa una solicitud a En proceso (rol Laboratorio).
registrarResultado()	Registra el resultado y completa la solicitud (RF-18).
guardarMedicamento()	Valida y guarda un medicamento: nombre único, precio > 0, stock ≥ 0 (RF-10).
entradaStock()	Suma unidades al inventario (RF-10).
comprar()	Compra del rol Paciente: valida stock y vencimiento, descuenta inventario y genera FAR-#### (RF-19, RN-06, RN-12).
crearFactura()	Crea una factura con monto > 0 (RF-11).
pagarFactura()	Registra el cobro con su método de pago (RF-11).
anularFactura()	Anula una factura pendiente con motivo (RN-07).
generarMensualidades()	Genera la mensualidad del mes sin duplicar (RN-08).
registrarDonacion()	Registra una donación monetaria o en especie (RF-11).
crearUsuario()	Crea una cuenta con contraseña temporal (RF-21, RN-09, RN-11).
cambiarActivo()	Activa o desactiva una cuenta (no la propia).
restablecerClave()	Asigna una nueva contraseña temporal a una cuenta (RF-21).
cambiarMiClave()	Cambia la contraseña propia validando la actual y la política (RF-22).
exportarRespaldo()	Descarga el respaldo JSON (RF-24).
restaurarRespaldo()	Restaura un respaldo JSON validado (RF-24).
cambiarMantenimiento()	Marca o quita el mantenimiento de una habitación (RF-20).
resumen()	Calcula los indicadores del panel y las alertas (RF-13).

7.2 js/auth.js · seguridad y sesión

Función	Qué hace
aHex()	Convierte un buffer de bytes en texto hexadecimal.
generarSal()	Genera una sal aleatoria de 16 bytes (32 caracteres hex).
hash()	Calcula el hash PBKDF2-SHA256 (100 000 iteraciones) de una contraseña con su sal; devuelve «pbkdf2$» + 64 hex.
claveValida()	Política de contraseña (RN-09): mínimo 8 caracteres, con letras y números.
leer()	Lee la sesión guardada en sessionStorage (o en memoria si no hay almacenamiento).
escribir()	Guarda o borra la sesión actual.

Función	Qué hace
registrar()	Agrega un registro a la bitácora de actividad (RF-23).
iniciarSesion()	Inicia sesión (RF-01, RF-02): valida en el servidor o, en demostración, contra el hash local; aplica el bloqueo de 3 intentos / 30 s y marca la contraseña temporal.
sesion()	Devuelve la sesión vigente o null si expiró por inactividad (30 min) o el usuario fue desactivado.
completarCambio()	Quita la marca de contraseña temporal de la sesión después de cambiarla.
tocar()	Renueva la hora de última actividad de la sesión.
cerrarSesion()	Cierra la sesión (RF-14): registra en bitácora, sincroniza pendientes y pide al servidor destruirla.

7.3 js/datos.js · acceso a datos

Función	Qué hace
pedir()	Envía una acción a la API (api.php) y devuelve la respuesta JSON con su código HTTP.
detectar()	Comprueba si hay servidor con base de datos (acción estado) para decidir el modo de trabajo.
vacia()	Crea la estructura vacía de colecciones en memoria.
foto()	Toma una copia de los datos para detectar cambios.
diferencias()	Calcula altas, cambios y bajas respecto a la última copia, para enviarlas al servidor.
cargarServidor()	Carga desde MySQL todos los datos permitidos para el rol (acción cargar).
sincronizarAhora()	Envía de inmediato los cambios pendientes al servidor (acción sincronizar).
cerrarServidor()	Limpia el estado del modo servidor al cerrar sesión.
pad()	Rellena un número con cero a la izquierda (fechas).
dia()	Devuelve la fecha de hoy desplazada «off» días, en formato AAAA-MM-DD.
cargar()	Carga los datos de demostración guardados en el navegador.
guardar()	Guarda los datos (navegador o sincronización con el servidor).
migrar()	Completa campos faltantes en datos de versiones anteriores.
exportar()	Genera el respaldo JSON completo (RF-24).
importar()	Valida y restaura un respaldo JSON (RF-24).
tabla()	Devuelve la colección indicada.
siguienteId()	Obtiene el siguiente identificador de una colección.
buscar()	Busca un registro por id.
insertar()	Inserta un registro y guarda.
actualizar()	Actualiza campos de un registro y guarda.
eliminar()	Elimina un registro por id y guarda.
eliminarDonde()	Elimina los registros que cumplen una condición.

Función	Qué hace
config()	Devuelve los parámetros de configuración del asilo.
datosIniciales()	Construye los datos ficticios de demostración (8 residentes, 20 habitaciones, 4 profesionales, 10 medicamentos).
res()	Ayudante: arma un residente de demostración.
ficha()	Ayudante: arma una ficha médica de demostración.
sembrar()	Carga los datos iniciales con las contraseñas temporales (solo su hash).
iniciar()	Inicializa la capa de datos al abrir la página.
reiniciar()	Restablece los datos iniciales (solo Administrador).

7.4 api/api.php · servidor

Función	Qué hace
responder()	Envía la respuesta JSON con su código HTTP y termina.
falla()	Responde un error con mensaje y código HTTP.
bd()	Devuelve la conexión PDO a MySQL (errores como excepción, utf8mb4).
entrada()	Lee el cuerpo JSON de la petición.
requiereSesion()	Exige sesión activa (401) y, salvo excepción, contraseña no temporal (403).
hashClave()	Calcula PBKDF2-SHA256 con 100 000 iteraciones (RNF-01).
nuevaSal()	Genera una sal aleatoria de 16 bytes.
claveValida()	Política de contraseña: 8+ caracteres con letras y números.
bitacora()	Inserta un registro en la tabla bitacora.
aBd()	Convierte un valor de la página al tipo de la columna MySQL.
deBd()	Convierte un valor de MySQL al tipo que usa la página.
upsert()	Inserta o actualiza una fila (INSERT … ON DUPLICATE KEY UPDATE).
filaSimple()	Mapea un objeto de la página a una fila de su tabla.
cargarTodo()	Lee todas las tablas y arma los datos que se envían a la página según el rol.
guardarResidente()	Guarda un residente validando en el servidor que la habitación exista, no esté en mantenimiento y tenga cama libre (HTTP 422, H-04).
guardarUsuario()	Guarda una cuenta; calcula el hash en el servidor y marca la contraseña temporal (H-02, H-03).
guardarCompra()	Guarda una compra de farmacia con su detalle.
borrar()	Elimina un registro por id (los residentes nunca, H-05).
sincronizar()	Aplica altas, cambios y bajas dentro de una transacción revisando permisos por tabla y rol (HTTP 403).
registrarFallo()	Cuenta un intento fallido y aplica el bloqueo de 30 s tras 3 intentos (RN-10).

7.5 js/app.js · presentación

Función	Qué hace
esc()	Escapa texto para insertarlo en HTML sin riesgo (XSS).
fmtQ()	Formatea un monto en quetzales.
fmtF()	Formatea una fecha AAAA-MM-DD como DD/MM/AAAA.
dateBox()	Caja de fecha (día y mes) para listas.
iniciales()	Iniciales de un nombre para el avatar.
horaAmPm()	Convierte HH:MM a formato a. m./p. m.
badge()	Insignia de estado con su color (mapa CLASE).
opciones()	Genera las opciones de un <select>.
residentesOpts()	Opciones con los residentes activos.
personalOpts()	Opciones con el personal activo.
datosForm()	Lee los campos de un formulario como objeto.
errorForm()	Muestra un error dentro del formulario.
vacioFila()	Fila de «sin resultados» para tablas.
coincide()	Indica si un texto coincide con la búsqueda actual.
heading()	Encabezado de página con título, subtítulo y botones.
ic()	Devuelve el ícono SVG indicado.
icono()	Ícono de un símbolo de texto.
stat()	Tarjeta de indicador con barra de progreso.
animar()	Anima los números de las tarjetas.
paso()	Paso de una animación de conteo.
panel()	Panel con título, herramientas y cuerpo.
tabla()	Tabla HTML con cabeceras.
persona()	Celda con avatar, nombre y subtítulo.
tabs()	Pestañas de navegación interna.
campo()	Campo de formulario con etiqueta.
input()	Control de texto.
select()	Control de selección.
pieForm()	Botones Cancelar / Guardar de un formulario.
toast()	Aviso emergente de éxito o error.
resultado()	Muestra el resultado de un servicio (aviso o error en el formulario).
abrirModal()	Abre una ventana modal.
cerrarModal()	Cierra la ventana modal.
confirmar()	Pide confirmación antes de una acción importante.

Función	Qué hace
formModal()	Abre un formulario en modal y procesa el envío.
verDocumento()	Vista previa de un documento imprimible.
imprimir()	Imprime un documento (o lo guarda como PDF).
docCab()	Encabezado de documentos impresos.
docPie()	Pie de documentos impresos.
exportarCSV()	Descarga una tabla como archivo CSV (RF-12).
ordenCita()	Ordena citas por fecha y hora.
saludo()	Saludo según la hora del día.
fechaLarga()	Fecha de hoy en texto largo.
filasResidentes()	Filas del listado de residentes con acciones.
actividad()	Elemento de próxima actividad.
proximasCitas()	Próximas citas filtradas.
accesos()	Tarjetas de acceso rápido del panel.
activarAccesos()	Asigna la navegación a los accesos rápidos.
lineaTiempo()	Agenda del día en línea de tiempo.
pct()	Posición porcentual de una hora en la línea de tiempo.
cumpleanos()	Próximos cumpleaños de residentes.
vInicio()	Vista Inicio del Administrador (RF-13).
listaPendientes()	Lista de pendientes del día.
vInicioMedico()	Vista Inicio del Médico.
vInicioLab()	Vista Inicio de Laboratorio (cola de trabajo).
vInicioPaciente()	Vista Inicio del Paciente.
vResidentes()	Vista Residentes: listado, búsqueda y filtros (RF-04, RF-07).
accionesResidente()	Maneja los botones de cada fila de residente.
formEgreso()	Formulario de egreso (RF-06).
formResidente()	Formulario de registro o edición de residente (RF-03, RF-05).
vFicha()	Ficha del residente: datos, signos, notas, medicamentos, citas y exámenes.
vital()	Tarjeta de un signo vital con alerta.
formSignos()	Formulario de signos vitales (RF-16).
formFicha()	Formulario de ficha médica (RF-15).
docFicha()	Ficha imprimible del residente.
vCitas()	Vista Citas médicas (RF-09).
formCita()	Formulario de nueva cita.
formReprogramar()	Formulario para reprogramar una cita.

Función	Qué hace
formCompletar()	Formulario para completar una cita con nota.
vPersonal()	Vista Personal médico (RF-08).
formPersonal()	Formulario de profesional.
vLaboratorio()	Vista Laboratorio clínico (RF-18).
formSolicitud()	Formulario de solicitud de examen.
formResultado()	Formulario de resultado de examen.
docResultado()	Documento imprimible de resultado.
verResultado()	Muestra el resultado de un examen.
vMedicamentos()	Vista Medicamentos: medicación del día e inventario (RF-10, RF-17).
formMedicamento()	Formulario de medicamento.
formEntrada()	Formulario de entrada de inventario.
comprobanteHtml()	Recibo de compra de farmacia FAR-####.
vFarmacia()	Vista Farmacia: catálogo y carrito (RF-19).
finalizarCompra()	Confirma la compra del Paciente.
vHabitaciones()	Vista Habitaciones: plano y mantenimiento (RF-20).
vCaja()	Vista Facturación (RF-11).
formMensualidades()	Formulario para generar mensualidades.
docRecibo()	Recibo de pago imprimible.
docConstancia()	Constancia de donación imprimible.
cajaFacturas()	Pestaña de facturas.
formFactura()	Formulario de nueva factura.
cajaDonaciones()	Pestaña de donaciones.
formDonacion()	Formulario de donación.
cajaResumen()	Pestaña de resumen de caja.
vReportes()	Vista Reportes con vista previa, PDF y CSV (RF-12).
enRango()	Indica si una fecha está en el rango del reporte.
tablaDoc()	Tabla para documentos impresos.
cajas()	Bloque de indicadores en reportes.
docReporte()	Arma el documento del reporte elegido.
vUsuarios()	Vista Usuarios: cuentas, bitácora y respaldo (RF-21, RF-23, RF-24).
formUsuario()	Formulario de nueva cuenta.
formMiClave()	Formulario para cambiar la contraseña propia (RF-22).
formClave()	Formulario para restablecer una contraseña.
vMis()	Vista Mi expediente del Paciente (RF-25).

Función	Qué hace
vRestringido()	Mensaje de acceso no permitido para el rol.
alCambiarRuta()	Reacciona al cambio de ruta (#/módulo).
ir()	Navega a una ruta.
navegar()	Dibuja el menú por rol y la vista de la ruta actual.
pintarNotificaciones()	Llena el panel de notificaciones.
pintarHeroStats()	Cifras públicas de la pantalla de inicio.
pintar()	Ayudante para dibujar las cifras de inicio.
mostrarCambioClave()	Muestra «Crea tu contraseña personal» (contraseña temporal).
mostrarLogin()	Muestra la pantalla de inicio de sesión.
cerrarMenu()	Cierra el menú lateral en móvil.
iniciarEventos()	Registra los eventos globales de la aplicación.
enviar()	Procesa el formulario de inicio de sesión.
pintarTema()	Actualiza el ícono del tema claro/oscuro.
aplicarTema()	Aplica el tema claro u oscuro.

8. Seguridad

- Contraseñas con PBKDF2-SHA256, 600 000 iteraciones y sal aleatoria de 16 bytes; se guarda «pbkdf2_sha256$600000$» + 64 hex (H-18). Los hashes del formato anterior «pbkdf2$» (100 000) se aceptan y se recalculan con sal nueva en el siguiente ingreso. Con servidor el hash lo calcula PHP (hash_pbkdf2; funciones claveCorrecta y requiereRehash).
- Mínimo privilegio (H-14): filtrarPorRol entrega a cada rol solo sus datos; validarEscrituraRol impide modificar registros ajenos (Paciente solo compras propias nuevas y descuento de stock; Médico sin ingresos ni egresos; Laboratorio sin crear solicitudes).
- Bitácora de solo agregar: guardarBitacora fija id, fecha, hora y usuario de la sesión; nadie la borra.
- Llaves foráneas ON DELETE RESTRICT (H-12): un registro con historial no se borra (422).
- Contraseñas iniciales o asignadas por el administrador marcadas como temporales (debe_cambiar_clave); el servidor responde 403 hasta que se cambian.
- Bloqueo de 30 s tras 3 intentos fallidos; cada intento fallido se registra en la bitácora con la IP.
- session_regenerate_id al iniciar sesión y destrucción de la sesión al salir (401 después).
- Consultas preparadas con PDO (evitan inyección SQL) y escape de HTML en la interfaz (función esc, evita XSS).
- Transacciones en cada sincronización: si algo falla no se guarda nada.
- En hosting, HTTPS obligatorio. Ningún documento contiene contraseñas.

9. Pruebas automatizadas

Los scripts de Evidencias/pruebas reproducen las 46 pruebas del 22/10/2026. El procedimiento completo, el cronograma y los resultados están en el documento «Plan de Pruebas e Informe de Verificación».

10. Mantenimiento

Tarea	Frecuencia	Cómo
Respaldo de la base	Semanal	phpMyAdmin › base asilo_cabeza_algodon › Exportar › SQL; guardar el archivo fuera del servidor
Respaldo JSON	Semanal	Usuarios › Descargar respaldo
Restaurar	Cuando se necesite	Importar el .sql en phpMyAdmin (o Restaurar el JSON en modo demostración). Probado en PR-07: 18 tablas con la misma suma de verificación tras restaurar
Actualizar una base existente	Al instalar la versión 3.1	Importar 04_migracion_v2.sql (05_ en hosting); no borra datos y al final muestra 0 reglas cascade
Monitoreo de disponibilidad	Continuo	Monitor externo gratuito (p. ej. UptimeRobot) a https://asilo-cabeza-algodon.site.je/api/api.php?accion=estado cada 5 min, de 07:00 a 18:00; ver Plan de Implementación
Revisar bitácora	Semanal	Usuarios › Bitácora de actividad: ingresos fallidos y acciones sensibles
Actualizar el sistema	Por versión	Respaldar, reemplazar la carpeta Sistema por la nueva versión etiquetada en GitHub, conservar config.php y probar el inicio de sesión
Actualizar XAMPP/PHP	Semestral	Respaldar primero; comprobar que PHP siga siendo 8.x

10.1 Solución de problemas técnicos

Síntoma	Causa probable	Solución
«No hay conexión con la base de datos (...)»	MySQL apagado o config.php incorrecto	Encender MySQL en XAMPP y revisar host, puerto, usuario, contraseña y nombre de base
Aviso «Modo demostración» aunque hay servidor	Se abrió el archivo directamente (file://) o la dirección lleva ?demo=1	Abrir por http://localhost/Sistema/ o por el enlace del hosting, sin ?demo=1
Aviso «Sin conexión con el servidor»	La API no respondió (Apache/PHP apagado, hosting caído o sin Internet)	Revisar Apache y MySQL o el estado del hosting y presionar «Reintentar»; la página no usa datos locales en ese caso
«No se puede eliminar: el registro tiene historial asociado»	Llave foránea RESTRICT (H-12)	Marcar el registro como inactivo en lugar de borrarlo
Error al importar 01_esquema.sql en hosting	El hosting no permite CREATE DATABASE	Usar los scripts de basedatos/hosting
Error al importar procedimientos en hosting	Sin permiso de rutinas	Omitir 04_procedimientos_opcional.sql; el sistema no los necesita para funcionar
«Este navegador no permite el cifrado seguro...»	Navegador antiguo o sitio sin https/localhost	Actualizar el navegador y abrir por https o localhost
HTTP 500 en api.php	Error de PHP o de la base	Revisar el registro de errores de Apache (xampp/apache/logs/error.log)

11. Control de versiones

El código y la documentación se publican en GitHub. Cada entrega se identifica con una versión y una etiqueta (por ejemplo v3.0.0). El procedimiento acordado con el SQA está en el documento «Protocolo de Entrega y Control de Versiones».